Инструмент структурного анализа и выявления аномалий в PE32+ AMD64 EH/Unwind-метаданных

Проблематика

EH/Unwind-метаданные определяют восстановление стека и участие обработчиков исключений, однако их ручная проверка требует многократных адресных преобразований и строгого соблюдения бинарного формата.

- Один PE-файл может содержать сотни или тысячи RUNTIME_FUNCTION. Проверка каждой записи вручную масштабируется линейно и повторяет одни и те же действия.
- Смешение VA, RVA и file offset приводит к чтению корректных байтов по неверному адресу и формирует последовательность ложных выводов.
- Неверный учёт дополнительных ячеек UWOP или выравнивания сдвигает чтение handler RVA и CHAININFO.
- Без фиксации поля, адреса, секции и точки отказа результат ручного анализа трудно воспроизвести и независимо проверить.

Аналитическая задача

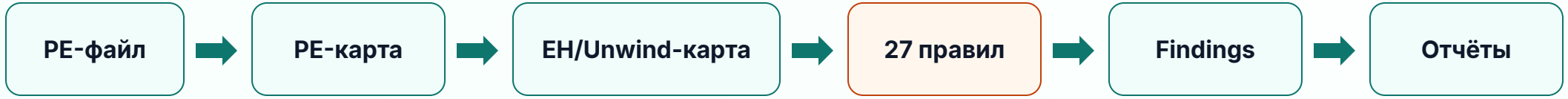
Операционной системе достаточно прочитать метаданные при выполнении. Аналитику требуется построить проверяемую карту: связать функцию, .pdata, UNWIND_INFO, секцию, права доступа и конкретное нарушение.

Искомый результат

Единообразная и воспроизводимая первичная проверка всего файла.

Решение

Инструмент статически разбирает PE32+ AMD64, строит функцию-ориентированную EH/Unwind-карту, проверяет 27 объяснимых правил и формирует отчёты. Windows-сборка дополнительно собирает ограниченные события выполнения.



Компонент результата	Назначение
Статическая карта	Связывает физические байты файла, секции, диапазоны функций и UNWIND_INFO.
Finding	Фиксирует сработавшее правило, серьёзность, количество нарушений и доказательные координаты.
Динамические данные	Подтверждают события, достигнутые в конкретном запуске под Windows Debug API.
Контракт отчёта	Передаёт одну и ту же модель в HTML, JSON, CSV и разметку Ghidra.

Сравнение с функциональными аналогами

Сравнение выполнено по стандартным возможностям, заявленным в официальной документации. Рассматриваемые средства решают смежные задачи и не являются полными взаимозаменяемыми реализациями.

Средство	Основное назначение	Работа с EH/Unwind	Динамическое наблюдение	Позиционирование относительно проекта
LLVM llvm-readobj	Вывод низкоуровневых структур объектных файлов	Вывод unwind information	Не предусмотрено	Декодирует данные, но не формирует каталог аномалий и объединённую оценку.
LIEF	Библиотека разбора и модификации исполняемых форматов	Модели x64 RUNTIME_FUNCTION и UNWIND_INFO	Не является готовым сценарием Windows-отладки	Предоставляет программный интерфейс; правила, отчёты и процесс валидации требуется создавать отдельно.
PE-bear	Графическое первичное исследование PE, включая повреждённые файлы	Инспекция в составе общей PE-карты	Не предусмотрено	Ориентирован на универсальный ручной обзор PE, а не на специализированную проверку EH/Unwind.
Ghidra	Дизассемблирование, декомпиляция, разметка и отладка	Использует данные в составе общего анализа программы	Через подсистему Debugger	Глубже исследует код; проект предварительно локализует записи и передаёт подготовленную разметку.
WinDbg	Интерактивная диагностика процесса Windows	Исключения, стек и точки останова	Полноценная интерактивная отладка	Сильнее в ручной динамической диагностике; не заменяет кроссплатформенный анализ файла и пакетные отчёты.
Данный проект	Специализированная проверка PE32+ AMD64 EH/Unwind	Карта функций, декодирование и 27 объяснимых правил	Ограниченная валидация через Windows Debug API	Единый воспроизводимый конвейер с HTML, JSON, CSV и экспортом в Ghidra.

Область применения

Продукт ориентирован на структурный анализ служебных данных Windows x64. Границы входа, режима выполнения и формируемого результата разделены явно.

Объект анализа	Статический режим	Динамический режим
Windows PE32+ AMD64 EXE, DLL и иные PE-образы	Работает без запуска файла. Ядро может быть собрано для Windows, Linux или macOS.	Работает только в Windows и создаёт исполняемый EXE как отлаживаемый процесс.

Предмет анализа

- Exception Directory и таблица RUNTIME_FUNCTION
- UNWIND_INFO, UNWIND_CODE, handler и CHAININFO

Результат

- Карта структур, локальные диагностики и 27 признаков
- HTML, JSON, CSV, Ghidra CSV и терминальная сводка

Назначение

- Первичная проверка и приоритизация ручного исследования
- Интеграция результатов в аналитический процесс

Архитектура проекта

Модульный монолит выполняет анализ в одном процессе. Модули обмениваются проверенными структурами данных и не дублируют интерпретацию недоверенных байтов.

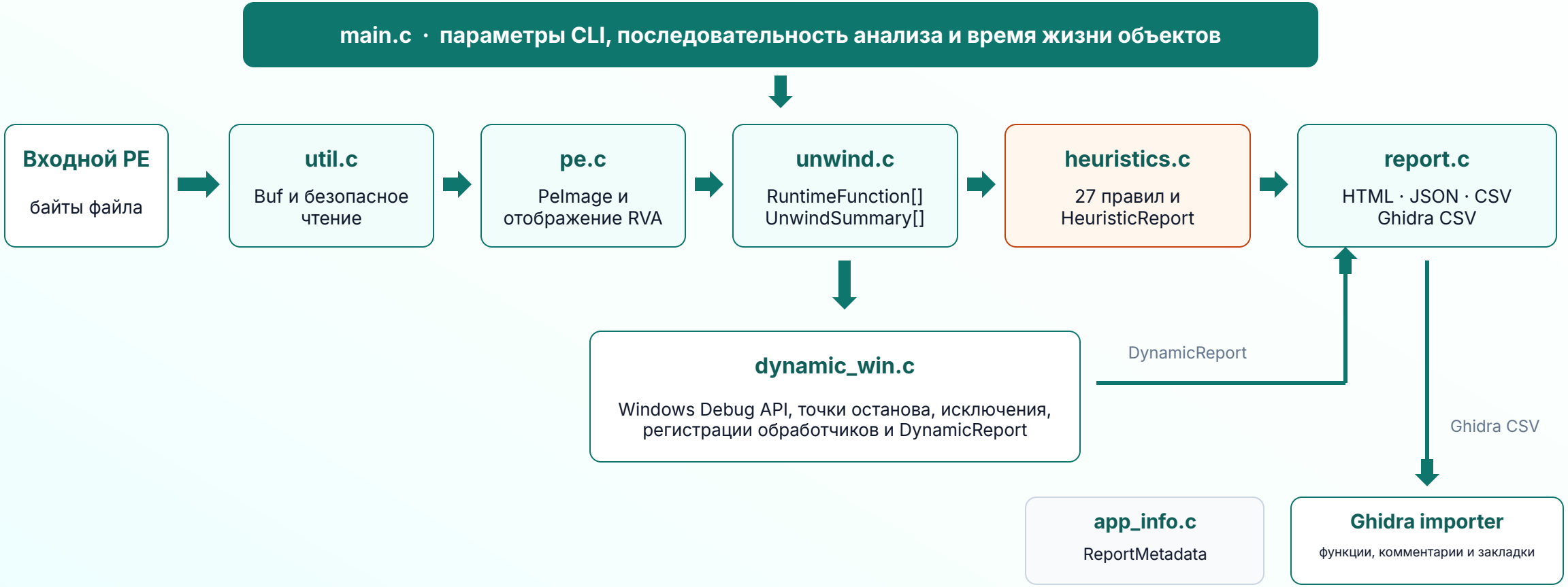
Модуль	Строгая ответственность
main.c	Разбор параметров, порядок вызовов, владение объектами и код завершения.
util.c	Загрузка Buf, безопасное чтение, строки, SHA-256 и атомарная запись.
pe.c	Допуск PE32+ AMD64, PeImage, секции и перевод RVA в file offset.
unwind.c	RUNTIME_FUNCTION, UNWIND_INFO, UWOP, handler и CHAININFO.
heuristics.c	27 правил, findings, сумма весов, Score, severity и confidence.

Модуль	Строгая ответственность
dynamic_win.c	Windows Debug API, breakpoints, события исключений и RIP trace.
report.c	Сериализация готовых моделей в терминал, JSON, HTML и CSV.
app_info.c	Версия, схема отчёта, платформа, компилятор и режим сборки.
include/*.h	Контракты структур и функций между реализациями модулей.
Ghidra importer	Перенос диапазонов, имён, комментариев и закладок в Ghidra.

Поток данных: Buf → PeImage → RuntimeFunction[] + UnwindSummary[] → HeuristicReport / DynamicReport → отчёты

Схема взаимодействия модулей

main.c управляет последовательностью вызовов. Статический маршрут строит проверенную EH/Unwind-карту, динамическая ветвь добавляет наблюдения Windows, а report.c формирует представления одной общей модели.



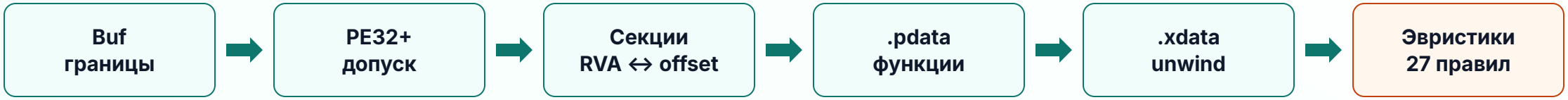
Технологический стек

Технологии выбраны по границам продукта: переносимое системное ядро, нативное наблюдение Windows, воспроизводимая сборка, машиночитаемые отчёты и автоматизированная проверка.

Уровень	Технологии	Назначение в проекте
Ядро	C17 · статическая библиотека ehua_core · CLI	Безопасный разбор байтов, PE-структур, unwind-метаданных и эвристическая оценка.
Сборка	CMake 3.23+ · CMake Presets · Ninja	Единые конфигурации разработки, тестирования, инструментирования и выпуска.
Компиляторы	MSVC 2022 · MinGW-w64/GCC · Clang/GCC	Проверка Windows x64 и переносимого статического ядра в Windows, Linux и macOS.
Системный слой	PE32+ AMD64 ABI · Windows Debug API	Статическое исследование формата и динамический сбор событий в Windows.
Отчётность	JSON Schema 2020-12 · HTML/CSS/JavaScript · CSV	Проверяемый контракт данных, самостоятельный HTML-отчёт и табличный экспорт.
Интеграция	Python 3 · Ghidra Script API	Импорт диапазонов функций, имён, комментариев и диагностических закладок.
Проверка качества	CTest · Python/jsonschema · ASan/UBSan · gcovr · libFuzzer · GitHub Actions	Модульные, регрессионные, схемные, мутационные и межплатформенные проверки.

Статический слой

Статический анализ исследует байты файла без передачи управления его точке входа. Каждый следующий этап использует только данные, прошедшие проверку границ и адресного отображения.



Уровень	Что проверяется	Что получает следующий этап
Форматный	MZ, PE signature, Machine AMD64, PE32+, размеры и таблица секций.	PeImage и единая система координат.
Синтаксический	RUNTIME_FUNCTION, заголовок UNWIND_INFO, UWOP, дополнительные slots, padding и tail.	UnwindSummary для каждой функции либо локальная диагностика.
Структурный	Границы, сортировка, перекрытия, CodeOffset, цепочки, адреса handler и права секций.	Счётчики нарушений и доказательные координаты.
Эвристический	Целостность, размещение, reuse, dominance и четыре комбинированных условия.	Findings, Severity, score_weight_sum, Score и Confidence.

Граница: версия не дизассемблирует .text и автоматически не сопоставляет UWOP с машинным прологом.

Динамический слой

Windows-сборка создаёт исследуемый процесс под Debug API и наблюдает только фактически достигнутые события. Статическая карта задаёт адреса интереса, а динамика добавляет сведения конкретного запуска.

Ветвь статических обработчиков

1. Адреса из статики

Для каждого доступного handler RVA вычисляется фактический адрес: runtime image base + RVA.

2. Точки наблюдения

На вычисленные адреса устанавливаются software breakpoints. Попадание формирует unwind_handler_hit.

3. Ближайший маршрут

После handler hit сохраняется ограниченная последовательность RIP. Это ближайшие адреса исполнения, а не полная трасса программы.

Ветвь системных API

1. Загруженные модули

После CREATE_PROCESS_DEBUG_EVENT обновляется список модулей и разрешаются экспортируемые адреса Windows API.

2. Контролируемые API

AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter, RtlAddFunctionTable и RtlInstallFunctionTableCallback.

3. Аргументы и события

Из регистров вызова фиксируются адреса callback или filter и параметры регистрации таблиц. Также сохраняются first-chance и second-chance исключения.

Интерпретация: вызов API подтверждает попытку регистрации, handler hit подтверждает достижение адреса. Отсутствие события в одном запуске не доказывает невозможность другого маршрута.

Семейства эвристических признаков

Каталог разделён на пять семейств по источнику доказательств. Такое группирование предотвращает ошибочное принятие нескольких связанных правил за независимые подтверждения.

Семейство	Правил	Предмет проверки	Аналитический вопрос
table_integrity	13	PE-секции, Exception Directory и RUNTIME_FUNCTION	Можно ли однозначно построить карту функций и адресов?
unwind_integrity	3	Доступность и внутренняя структура UNWIND_INFO	Декодируются ли метаданные без противоречий и обрывов?
metadata_placement	7	Секция и права UNWIND_INFO и handler	Соответствует ли размещение ожидаемой роли данных и кода?
unwind_reuse	3	Повторное использование UnwindInfoAddress	Не концентрируется ли большое число функций на общей metadata?
handler_dominance	1	Распределение адресов обработчиков	Не обслуживает ли один handler аномально большую долю функций?

Комбинированные признаки добавляют небольшой вес за согласование сигналов, но не заменяют исходные findings и не считаются полностью независимыми доказательствами.

27 диагностических признаков

Каждый диагностический признак формализует отдельное условие эвристического анализа. Его формулировка характеризует установленное структурное отклонение, не определяя его происхождение и не являясь самостоятельным свидетельством вредоносности файла.

1. pe_section_raw_oob
файловый диапазон секции выходит за файл

2. pe_section_virtual_oob
виртуальный диапазон секции выходит за образ

3. pe_section_raw_overlap
файловые диапазоны секций пересекаются

4. pe_section_virtual_overlap
виртуальные диапазоны секций пересекаются

5. pe_section_misaligned
нарушено выравнивание секции

6. pe_exception_directory_oob
Exception Directory недоступна или неполна

7. no_runtime_functions
есть исполняемый код, но нет таблицы функций

8. invalid_ranges
EndRVA не больше BeginRVA

9. rf_oob
диапазон функции выходит за SizeOfImage

10. unwind_rva_oob
UnwindInfoAddress не разрешается в доступные байты

11. rf_unsorted
таблица функций не отсортирована по BeginRVA

12. rf_overlaps
диапазоны функций перекрываются

13. duplicate_begin
несколько функций имеют одинаковый BeginRVA

14. unwind_parse_fail
UNWIND_INFO невозможно безопасно декодировать

15. unwind_anomalies
поля или операции UNWIND_INFO противоречивы

16. unwind_reuse_moderate
умеренное повторное использование unwind metadata

17. unwind_reuse_strong
массовое повторное использование unwind metadata

18. handler_dominance
один handler доминирует среди функций

19. unwind_in_exec_section
UNWIND_INFO находится в исполняемой секции

20. unwind_unusual_section
UNWIND_INFO находится в нетипичной секции

21. unwind_writable_section
UNWIND_INFO находится в записываемой секции

22. handler_nonexec_section
handler не разрешается в исполняемую секцию

23. handler_writable_section
handler расположен в записываемой секции

24. placement_exec_nonexec_handler_combo
исполняемая metadata сочетается с non-exec handler

25. placement_integrity_combo
нетипичное размещение сочетается с повреждением

26. reuse_dominance_combo
reuse сочетается с доминированием handler

27. writable_metadata_handler_combo
metadata и handler одновременно записываемы

Интерпретация результатов анализа

Результат формируется последовательно: наблюдение превращается в finding, после чего отдельно рассчитываются серьёзность, суммарная оценка и достаточность доказательной базы.

Score

score_weight_sum = сумма базовых весов всех активных findings.

Score = min(score_weight_sum, 100).

Пример: сумма 176 сохраняется полностью, итоговый Score равен 100.

Severity

Характеристика отдельного finding.

medium: локальное или контекстное отклонение с возможным допустимым объяснением.

high: жёсткое нарушение структуры, доступности или однозначности карты.

Confidence

Характеристика итогового статического вывода.

0,15 + 0,11 × семейства + 0,08 × high + 0,04 × medium + бонусы − ослабления.

Результат ограничивается диапазоном 0,05–0,90.

Score 0–14	Score 15–39	Score 40–69	Score 70–100
Сильные статические аномалии не установлены	Низкая подозрительность	Умеренная подозрительность	Высокая подозрительность

Score задаёт приоритет ручной проверки и не является вероятностью вредоносности. Состав причин раскрывают findings, evidence и матрица 27 правил.

Отчёты

report.c не анализирует файл повторно. Модуль сериализует уже построенные PE-, EH/Unwind-, эвристические и динамические модели в представления для разных способов использования.

Формат	Содержание	Назначение
Терминал	Статус запуска, основные счётчики и итоговые оценки.	Быстрый контроль одного запуска.
JSON 1.3.0	Полная машиночитаемая модель, SHA-256, функции, findings, score_weight_sum, Score и динамика.	Backend, база данных, сравнение и автоматическая обработка.
HTML	Самодостаточный интерактивный отчёт: сводка, PE-карта, RUNTIME_FUNCTION, 27 правил, evidence и runtime events.	Исследование результата человеком без внешних ресурсов.
CSV	Плоские строки функций и сводных полей UNWIND_INFO.	Табличная обработка и фильтрация.
Ghidra CSV	Диапазоны, допустимые имена, комментарии и диагностические признаки.	Создание или уточнение разметки в Ghidra.

Доказательная цепочка отчёта: PE → Exception Directory → .pdata entry → функция → UnwindInfoAddress → поле или агрегированное условие → finding.

Тестирование

Проверка разделяет корректность функций, воспроизводимость известных сценариев, устойчивость к повреждённому вводу и поведение Windows-коллектора. Один показатель не заменяет остальные уровни.

Уровень	Проверочный объём	Что подтверждается
Автоматизированный CTest	19 из 19 на текущей исходной базе	Модульные, регрессионные, схемные и семантические проверки; контракт CLI, входная валидация и отчёты.
Синтетические PE	84 файла: 66 статических, 9 динамических, 9 смешанных	Детерминированные нарушения полей, структуры и сочетаний признаков.
Мутационное тестирование	120 мутаций в текущем CTest-сценарии	Контролируемый отказ или корректный отчёт без аварийного завершения.
Доброкачественный корпус	106 PE-файлов в зафиксированной Windows-проверке	Работа на реальных компиляторных образцах; ненулевые статические Score не зафиксированы.
Динамический корпус Windows	124 отчёта; 86 динамических сессий запущено	Состояния сборщика Windows Debug API, события выполнения и отсутствие неожиданных сбоев.
Отчётные контракты	JSON Schema 1.3.0 и межполевые проверки	Score = min(score_weight_sum, 100), координаты, статусы и согласованность представлений.

Синтетические образцы дают точную причинность, реальные файлы проверяют применимость, а Windows-сценарии подтверждают только фактически выполненные маршруты.

Практическая значимость

Инструмент автоматизирует повторяемую часть EH/Unwind-исследования и передаёт аналитику структурированную основу для дальнейшей проверки машинного кода и поведения процесса.

Практический эффект	Конкретный результат
Обработка больших таблиц	Последовательный разбор всех доступных RUNTIME_FUNCTION вместо ручной проверки единичных записей.
Снижение адресных ошибок	Единый перевод RVA в file offset, проверка границ Buf и локализация точки отказа.
Воспроизводимость	SHA-256, параметры запуска, версия схемы и доказательные координаты сохраняют условия анализа.
Приоритизация	Severity и Score выделяют существенные записи, а confidence показывает достаточность совокупной доказательной базы.
Интеграция	HTML поддерживает ручное изучение, JSON — серверную обработку и хранение, CSV — табличный анализ, Ghidra CSV — рабочую разметку.
Аудит вывода	Матрица 27 правил и карточки findings позволяют проверить основание каждого аналитического заключения.

Ценность проекта состоит в построении проверяемой EH/Unwind-карты, а окончательная интерпретация остаётся задачей специалиста.

Граница продукта

Явные ограничения отделяют реализованную структурную проверку от задач, которые требуют дизассемблирования, знания конкретного языкового runtime или более полного наблюдения процесса.

Не выполняется

Полная декомпиляция и восстановление C/C++

Универсальная классификация вредоносного ПО

PE32 x86, ARM64, ELF и Mach-O

Трассировка режима ядра и эмуляция всех путей

Выполняется частично

Нет автоматического сопоставления UWOP с прологом

Нет универсального разбора языково-зависимых таблиц

RtlAddFunctionTable фиксируется без полного разбора таблицы в памяти процесса

Трасса RIP ограничена ближайшими адресами после попадания

Условия применения

Объект статики: Windows PE32+ AMD64

Статическое ядро: Windows, Linux или macOS

Динамический режим: только исполняемый PE-файл под Windows

Динамический вывод относится только к фактически выполненному сценарию

Ограничения входят в контракт продукта и не должны подменяться более широкими выводами отчёта.

Дальнейшее развитие

Развитие ориентировано на усиление доказательной связи между метаданными, машинным кодом и наблюдаемым выполнением без изменения модульных границ ядра.

Приоритет	Направление	Техническое изменение	Ожидаемый результат
1	UWOP и пролог .text	Добавление дизассемблерного слоя и сопоставление эффектов стека и регистров.	Автоматическая фиксация совпадений и расхождений метаданных с инструкциями.
2	Языково-зависимые данные	Декодирование известных структур, начиная с таблиц областей __C_specific_handler.	Защищённые диапазоны, фильтры и цели обработчиков в отчёте.
3	Динамические таблицы функций	Чтение памяти процесса и проверка записей, переданных RtlAddFunctionTable.	Переход от фиксации вызова к проверке зарегистрированной карты времени выполнения.
4	Хранение и сравнение	Серверный слой, база данных, версионированная схема JSON и сопоставление запусков.	История образцов, сравнение версий и повторяемые аналитические выборки.

Базовая архитектура сохраняется: новые анализаторы добавляют доказательные данные в существующие модели, а report.c продолжает отвечать только за представление результата.

Спасибо за внимание

Инструмент структурного анализа и выявления аномалий в PE32+ AMD64 EH/Unwind-метаданных